

Engineering Assignment

Relational Database Design & ERD Documentation

Presented by: Omar Althuniyan

Summary

1. Introduction & ERD Concept Overview	2
2. Retail Business Entity-Relationship Diagram (ERD)	3
3. Entities and Attributes Description	4
4. Relationship Logic & Cardinality	8
5. Engineering Justification & Best Practices	8
6. Conclusion	9

Introduction & ERD Concept Overview

1.1 Overview

An Entity-Relationship Diagram (ERD) is an effective tool in database engineering used to visualize the structure of a database and the logical relationships between different data objects. In the context of a retail business, an ERD helps in organizing complex information such as customer profiles, product inventories, and sales transactions into a coherent and efficient system.

1.2 Project Objectives

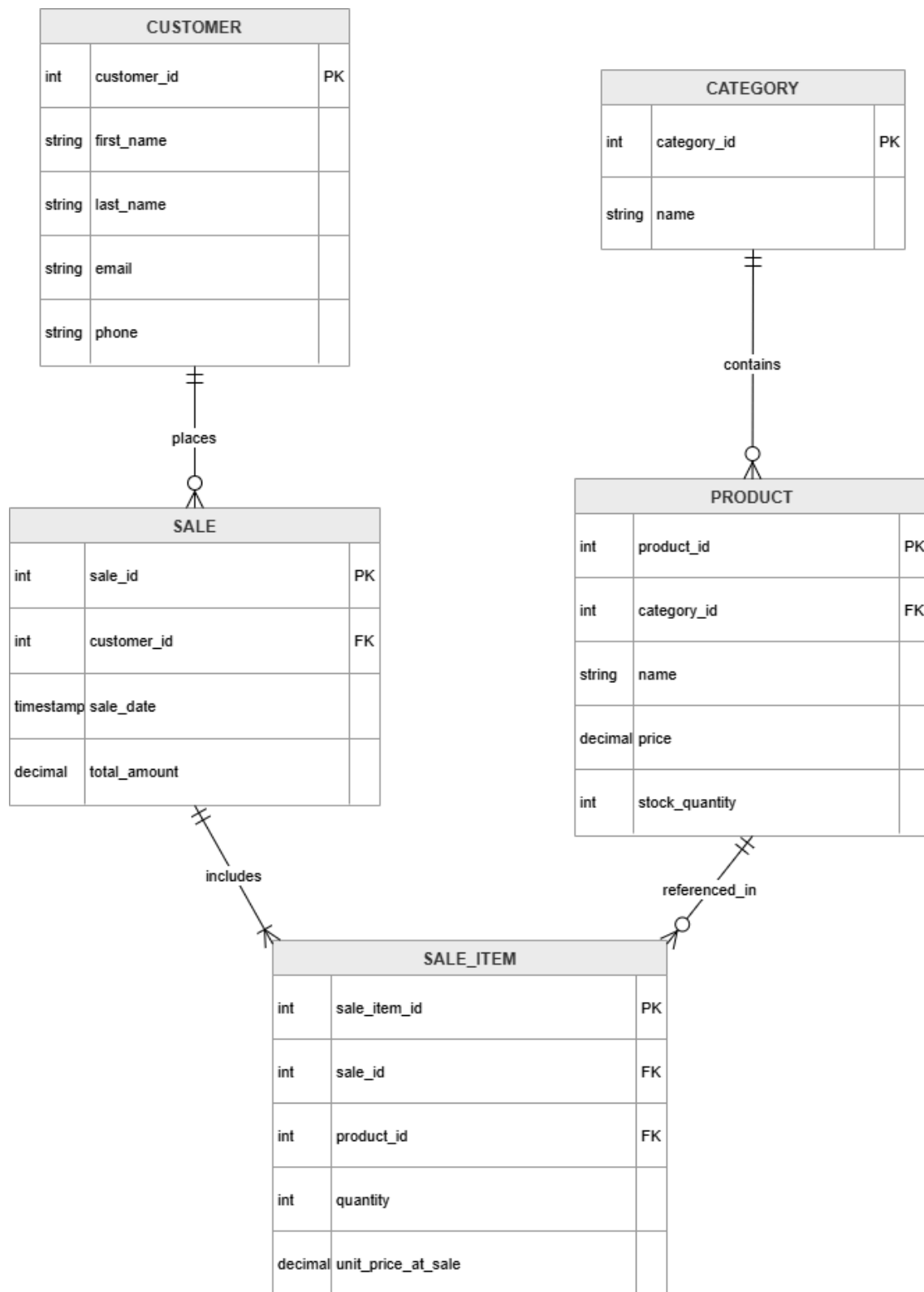
The primary goal of this model is to design a database foundation for a small retail business. This involves:

- **Data Consistency:** Ensuring that every sale is linked to a valid customer and product.
- **Normalization (3NF):** Reducing data redundancy to optimize storage and performance.
- **Transaction Accuracy:** Capturing detailed sales information, including quantities and historical pricing.

1.3 Design Methodology

The design follows the **Relational Database Management System (RDBMS)** principles. By identifying the core entities—**Customers, Products, Categories, Sales**—we have established a structure that supports daily operations while remaining flexible enough for future scalability and reporting.

Retail Business Entity-Relationship Diagram (ERD)



Entities and Attributes Description

This section provides a detailed breakdown of the entities defined in the ERD, their respective attributes, and the purpose of each within the retail database system.

3.1 CUSTOMER Entity

Contains information regarding the individuals who interact with the business.

- **customer_id (PK):** A unique identifier for each customer.
- **first_name / last_name:** Used for personalized interaction and identification.
- **email:** Set as a **Unique** field to prevent duplicate accounts.
- **phone:** Primary contact number for order notifications.

3.2 CATEGORY Entity

Designed to organize products into logical groups (e.g., Electronics, Apparel).

- **category_id (PK):** Unique identifier for each category.
- **name:** The descriptive label for the category.
- *Purpose:* Enhances inventory management and allows for category-based sales reporting.

3.3 PRODUCT Entity

Represents the items available for sale in the retail store.

- **product_id (PK):** Unique identifier for each product.
- **category_id (FK):** Links the product to its specific category.
- **name:** The commercial name of the item.
- **price:** The current listing price of the product.
- **stock_quantity:** Tracks the available units in the inventory.

3.4 SALE Entity

Acts as the "Header" for each transaction made by a customer.

- **sale_id (PK):** Unique identifier for each sales transaction.
- **customer_id (FK):** Identifies which customer placed the order.
- **sale_date:** Captures the exact timestamp of the transaction.
- **total_amount:** The final calculated value of the entire sale.

3.5 SALE_ITEM Entity (Junction Table)

This is a critical entity that handles the many-to-many relationship between Sales and Products.

- **sale_item_id (PK):** Unique identifier for each line item in a receipt.
- **sale_id (FK) / product_id (FK):** Linkages to the respective sale and product records.
- **quantity:** The number of units purchased for a specific product.
- **unit_price_at_sale: (Crucial Field)** Captures the price of the product at the exact moment of sale. This ensures historical financial accuracy even if product prices are updated in the master table later.

Relationship Logic & Engineering Justification

Relationship Logic & Cardinality

The model utilizes **Crow's Foot Notation** to define the interaction between entities. The following logic ensures data consistency:

1. **CATEGORY to PRODUCT (1:N):** One category can contain multiple products, but each product belongs to only one category. This hierarchy simplifies inventory management.
 2. **CUSTOMER to SALE (1:N):** A single customer can place multiple orders over time. This relationship allows the business to track customer purchase history for loyalty programs.
 3. **SALE to SALE_ITEM (1:N):** Each sale (receipt) is composed of one or more line items. This structure supports "multi-product" transactions.
 4. **PRODUCT to SALE_ITEM (1:N):** A single product can appear in many different sales transactions across the system.
-

Engineering Justification & Best Practices

To ensure this model meets professional engineering standards, the following principles were applied:

- **Third Normal Form (3NF) Compliance:** By separating categories and sales details into distinct entities, we have eliminated data redundancy. This prevents anomalies and ensures that an update in one table doesn't lead to inconsistencies elsewhere.
- **Resolution of Many-to-Many Relationships:** Products and Sales have a natural many-to-many relationship. We successfully resolved this by introducing the **SALE_ITEM** junction table, which is the standard approach for scalable database design.
- **Historical Data Integrity:** Unlike basic models, this design includes the `unit_price_at_sale` field. This is a critical business logic feature that preserves the price at the time of purchase, ensuring that financial reports remain accurate even if the product's master price is adjusted in the future.

Conclusion

In conclusion, the proposed Data Model for the retail business provides a robust, reliable, and highly scalable foundation for database management. By adhering to the principles of the **Third Normal Form (3NF)** and resolving complex many-to-many relationships through a structured **Junction Table**, the design ensures maximum data integrity and prevents common data redundancies.

Furthermore, the inclusion of **historical price tracking** at the transaction level demonstrates a professional approach to financial accuracy, ensuring the system remains consistent over time. This model is fully optimized for implementation in any modern **Relational Database Management System (RDBMS)** and is prepared to support future business intelligence, reporting, and analytical needs.